

Programa de Ingeniería de Sistemas y Computación

Universidad del Quindío

Curso: Programación Avanzada

Guía: 14

Título: CONFIGURACIÓN ANGULAR

Duración estimada: 120 minutos

Docentes: Christian Andrés Candela

OBJETIVO

Construir y configurar la primera versión del proyecto frontend usando el framework Angular. Esta versión incluirá la creación de componentes básicos, configuración de rutas y una estructura inicial adecuada para el desarrollo de aplicaciones web modernas.

CONCEPTOS BÁSICOS

- Lenguaje HTML: Estructura básica de páginas web.
- Hojas de estilo CSS: Personalización visual de interfaces.
- TypeScript: Superset de JavaScript que añade tipado estático y características avanzadas.
- Angular CLI: Interfaz de línea de comandos para gestionar proyectos Angular.

CONTEXTUALIZACIÓN TEÓRICA

Angular 21 (<https://angular.dev/>) es un marco de trabajo (framework) de código abierto desarrollado por Google, diseñado para construir aplicaciones web dinámicas y de una sola página (SPA). Su arquitectura se basa en el patrón Modelo-Vista-Controlador (MVC), donde:

- **Modelo:** Gestiona los datos y su estructura.
- **Vista:** Define la interfaz de usuario mediante plantillas HTML que incorpora directivas de Angular.
- **Controlador:** Implementado como componentes TypeScript, encapsula la lógica de la vista y los datos.

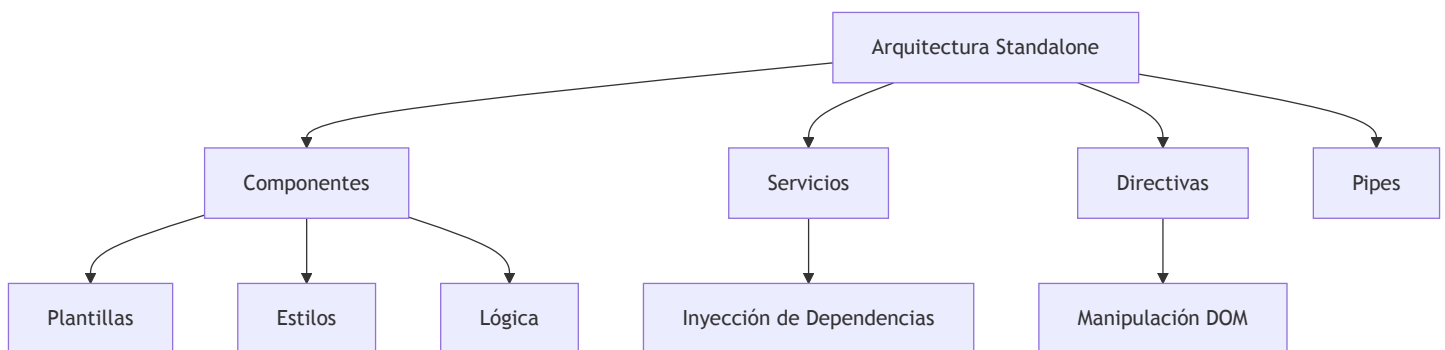
Características Clave de Angular

- **Arquitectura basada en componentes:** Bloques fundamentales que combinan lógica, plantillas HTML y estilos CSS. Los componentes pueden anidarse para formar una jerarquía.
- **Directivas y Control de Flujo:** Manipulan el DOM de manera dinámica. En Angular moderno se recomienda usar la sintaxis de bloque integrada (`@if` , `@for`), aunque históricamente se usan directivas estructurales (`*ngIf` , `*ngFor`). También es posible crear directivas personalizadas.
- **Servicios:** Clases TypeScript que organizan y comparten lógica reutilizable, como lógica de negocio o acceso a datos.
- **Inyección de Dependencias:** Facilita la creación, gestión e intercambio de dependencias entre diferentes partes de la aplicación.
- **Routing:** Permite la navegación entre diferentes vistas (componentes) de una aplicación SPA.
- **Señales (Signals):** El núcleo de la reactividad moderna en Angular 21. Permiten a la vista actualizarse automáticamente con precisión quirúrgica cuando cambian los datos subyacentes, sin requerir recargas masivas.

Funcionamiento

Angular incluye una **interfaz de línea de comandos** (CLI) que facilita la creación, desarrollo y mantenimiento de aplicaciones.

A nivel general podemos ver la arquitectura del funcionamiento de Angular así:

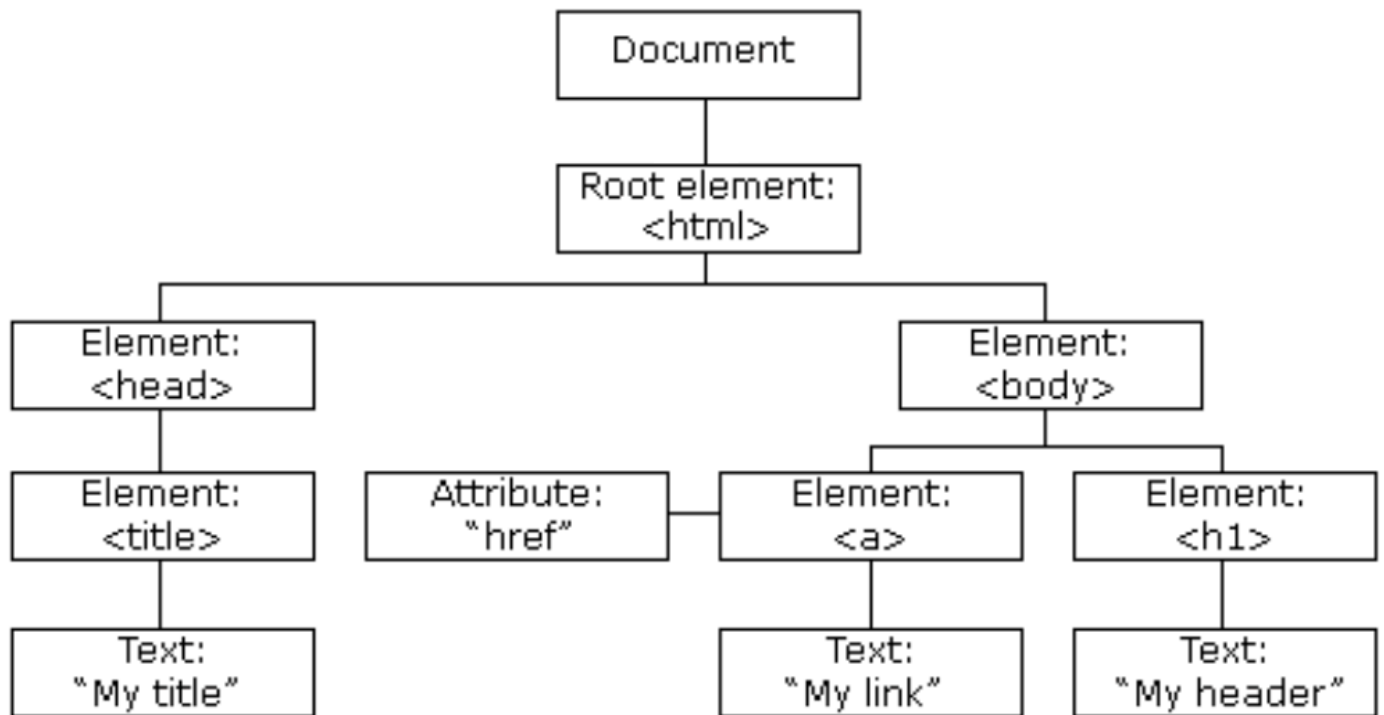


Aplicaciones web de una sola página (SPA)

Las SPAs cargan una única página HTML y actualizan dinámicamente su contenido según las interacciones del usuario. Algunos ejemplos incluyen Gmail, Twitter y Google Maps.

DOM (Document Object Model)

El DOM es una representación en árbol de la estructura de un documento HTML, que permite manipular dinámicamente el contenido, estructura y estilos de una página web.



Para más información:

<https://www.hiberus.com/crecemos-contigo/que-es-angular-y-para-que-sirve/>

PRECAUCIONES Y RECOMENDACIONES

- Asegúrate de tener instalado Node.js en su última versión.
- Verifica que Angular CLI esté correctamente instalado ejecutando `ng version`.
- No es necesario tener el backend configurado para esta guía.

ARTEFACTOS

- Visual Studio Code en su última versión.
- [WebStorm](#) (Como alternativa a Visual Studio).
- Controlador de versiones GIT.

EVALUACIÓN O RESULTADO

Se espera que el estudiante comprenda:

1. La utilidad de Angular y sus conceptos fundamentales.
2. La estructura básica de un proyecto Angular.
3. Cómo crear componentes y configurar rutas.

Procedimiento

1. Instalación de Herramientas

1. Instalar NodeJS y npm:

Descargue desde [Node.js](https://nodejs.org/).

Verifica la instalación:

```
node -v  
npm -v
```

2. Instalar Angular CLI:

Ejecute en la terminal:

```
npm install -g @angular/cli
```

Para comprobar la versión:

```
ng version
```

2. Creación del Proyecto Angular

1. Ejecuta el siguiente comando para crear un nuevo proyecto:

```
ng new my-app
```

- Habilita Angular routing (`y`).
- Elige CSS como formato de hojas de estilos.
- Deshabilita SSR y Static Site Generation (`N`).

- Opcionalmente, usa este comando simplificado para usar la configuración por defecto de Angular:

```
ng new my-app --defaults --standalone=true
```

3. Estructura del Proyecto (Standalone)

1. Abrir el proyecto en Visual Studio Code:

La estructura inicial será similar a esta:

```
/
├ .vscode/
├ node_modules/
├ public/
├ src/
│   ├── app/
│   │   ├── app.css
│   │   ├── app.html
│   │   ├── app.ts
│   │   ├── app.config.ts
│   │   └── app.routes.ts
│   ├── index.html
│   ├── main.ts
│   └── styles.css
├ angular.json
├ package.json
└ tsconfig.json
```

2. Descripción de algunos elementos:

- `.vscode`: Contiene archivos propios del IDE como configuraciones y variables de entorno.
- `node_modules`: Contiene todas las dependencias usadas en el proyecto.
- `public`: Archivos estáticos de estilos, imágenes, iconos, fuentes, etc.
- `src`: Contiene los archivos de código fuente del proyecto.
 - `index.html`: Página web principal de la aplicación.
 - `styles.css`: Contiene los estilos globales de la aplicación.
 - `main.ts`: es el punto de entrada de la aplicación Angular. Cuando se inicia la aplicación, este archivo se ejecuta primero. Su función principal es inicializar la aplicación y cargar el módulo raíz (generalmente llamado `AppModule`) o componente raíz (`AppComponent`).
 - `app`: Es donde programaremos los componentes, servicios y estructura de los documentos HTML. También será donde programaremos la lógica necesaria para que

los datos se carguen en el frontend desde el backend o para procesar los formularios, esta parte se debe programar en el lenguaje de programación TypeScript.

- App (Componente): Es el componente raíz de una aplicación Angular. Un componente puede representar toda una página web completa o parte de esta. Por lo general, cada componente está compuesto por un archivo ts (Controlador), html (Vista) y css (estilos). El nombre de los archivos pertenecientes al componente inician con **app** así:
 - app.css: Contiene los estilos de la página del componente. Estos estilos se definen por medio del lenguaje CSS (Cascading Style Sheets).
 - app.html: Contiene la estructura visual del componente. Esta estructura se escribe usando el lenguaje HTML, el cual es un lenguaje de etiquetas de hipertexto.
 - app.spec.ts: Contiene las implementaciones de las pruebas unitarias.
 - app.ts: Contiene el código escrito en lenguaje TypeScript cuyo propósito es gestionar la información y escuchar los eventos que ocurren en la página html.
- app.routes.ts: Es el archivo donde se definen las rutas de la aplicación así como sus restricciones, parámetros, entre otras.
- app.config.ts: Es el archivo donde se definen los elementos globales de configuración de la aplicación.
- package.json: es la configuración de nuestra aplicación. Contiene el nombre de la app, las dependencias necesarias para su correcta ejecución y muchas otras cosas.

Para más información:

- <https://medium.com/notasdeangular/componentes-en-angular-f25138b00c83>
- <https://danielherresan.medium.com/arquitectura-limpia-en-angular-17-25c129ed2f7a>
- <https://imaginaformacion.com/tutoriales/aprende-angular-7-tutorial-de-primeros-pasos>

4. Ejecución de la aplicación

1. Ejecuta el siguiente comando para iniciar el servidor de desarrollo:

```
npm start
```

Nota: Se utiliza `npm start` (en lugar del clásico `ng serve`) porque este comando garantiza que se utilice la versión del Angular CLI instalada localmente dentro de tu carpeta `node_modules`, previniendo errores de "comando no encontrado" si no tienes permisos globales en tu sistema operativo.

La aplicación estará disponible abriendo manualmente en tu navegador la ruta `http://localhost:4200` .

5. Creación de Componentes Básicos

1. Crea directorios de trabajo dentro de `src/app` :

- `dto` : Para clases/interfaces de datos.
- `componentes` : Para los componentes visuales.
- `servicios` : Para la lógica de negocio y conexión con APIs.

```
/
└─ src/
   └─ app/
      ├── componentes/
      ├── dto/
      └── servicios/
```

2. Genera componentes básicos:

```
ng generate component componentes/inicio --skip-tests
ng generate component componentes/login --skip-tests
ng generate component componentes/registro --skip-tests
```

Tras la ejecución de estos comandos se deben haber creado 3 sub carpetas dentro de `componentes` (`inicio`, `login` y `registro`). Cada una de ellas representando un componente.

3. Estructura de un componente. Cada componente se ha creado en una carpeta, la cual contiene los archivos `css`, `html` y `ts`. En Angular 21, los archivos de componentes ya no llevan el sufijo `.component` . Abra cada uno de los archivos (ej. `inicio.ts` , `inicio.html` , `inicio.css`) y analice su contenido.

- Archivo `.ts`: Contiene la lógica y la configuración del componente. La clase exportada tiene un nombre simple (ej. `export class Inicio`).
- Archivo `.html`: Define la estructura visual (vista) del componente. Muestra los datos al usuario y permite la interacción mediante eventos (clicks, formularios, etc).
- Archivo `.css`: Contiene los estilos específicos del componente.

Relación entre los archivos

Todos los archivos están vinculados a través del decorador `@Component` en el archivo `.ts`. `@Component` tiene las propiedades `selector` que es el nombre del componente, `templateUrl` que es el archivo `html`, `styleUrl` que es la hoja de estilo propia del componente e `imports`

que es donde se definen componentes, módulos o librerías externas que serán usadas dentro del componente.:

```
@Component({
  selector: 'app-mi-componente',
  imports: [],
  templateUrl: './mi-componente.html', // Vínculo al HTML
  styleUrls: ['./mi-componente.css'] // Vínculo al CSS
})
```

6. Modificando la vista

1. Modificar el archivo `src/app/app.html` :

Para empezar establezca un encabezado y un pie de página así:

```
<header>
  <p>Página inicial de {{ title() }}</p>
</header>
<router-outlet />
<footer>Universidad del Quindío</footer>
```

Observe que se está inyectando una variable `{{title()}}` en el archivo html que viene desde el controlador (`app.ts`). Al estar en Angular 21, usamos **Señales** (Signals), por lo que debemos llamar a la variable con paréntesis () para leer su valor dinámico.

`<router-outlet>` es un componente de Angular que permite reemplazar su contenido según la ruta que definamos. Por ahora el router-outlet está vacío ya que aún no hemos configurado rutas en la aplicación.

2. Crear variables:

Modifique el archivo `src/app/app.ts` adicionando una nueva variable footer con el valor "Universidad del Quindío" así:


```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  title = signal('My-App');
  footer = signal('Universidad del Quindío');
}
```

3. Modificar el archivo `src/app/app.html` :

Haga uso de la nueva variable en su código html así:

```
<header>
  <p>Página inicial de {{ title() }}</p>
</header>
<router-outlet />
<footer>{{ footer() }}</footer>
```

7. Configuración de Rutas

1. Modifica `src/app/app.routes.ts` para definir las rutas:

```
import { Routes } from '@angular/router';
import { Inicio } from '../componentes/inicio/inicio';
import { Login } from '../componentes/login/login';
import { Registro } from '../componentes/registro/registro';

export const routes: Routes = [
  { path: '', component: Inicio },
  { path: 'login', component: Login },
  { path: 'registro', component: Registro },
  { path: '**', pathMatch: 'full', redirectTo: '/' },
];
```

2. Actualiza `src/app/app.html` para incluir un menú de navegación:

```

<header>
  <p>Página inicial de {{ title() }}</p>
  <ul>
    <li><a href="">Inicio</a></li>
    <li><a href="/login">Login</a></li>
    <li><a href="/registro">Registro</a></li>
  </ul>
</header>
<router-outlet />
<footer>{{ footer() }}</footer>

```

Pruebe dando click en cada link y observe cómo cambia la url y el contenido de la página de acuerdo a la opción elegida. El componente router-outlet está cambiando en función de la página cargada, como cada ruta está asociada a un componente entonces el router-outlet se trae el archivo html de dicho componente.

3. Actualiza `src/app/app.html` usando `routerLink` en lugar de `href`. `routerLink` se utiliza para crear enlaces de navegación en la aplicación Angular. Permite que los usuarios de la aplicación naveguen de una vista a otra haciendo clic en los enlaces, y Angular maneja el enrutamiento de la aplicación y carga la vista correspondiente sin recargar la página totalmente (solo actualiza el componente que sea necesario).

```

<header>
  <p>Página inicial de {{ title() }}</p>
  <ul>
    <li><a routerLink="/">Inicio</a></li>
    <li><a routerLink="/login">Login</a></li>
    <li><a routerLink="/registro">Registro</a></li>
  </ul>
</header>
<router-outlet />
<footer>{{ footer() }}</footer>

```

4. Importa `RouterLink` en `src/app/app.ts` :

```
import { Component, signal } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  title = signal('My-App');
  footer = signal('Universidad del Quindío');
}
```

8. Avance del proyecto final

1. Cree los componentes que crea necesarios para su proyecto final y añada las rutas correspondientes en `app.routes.ts`.
2. Cree un nuevo repositorio en github y suba el código.

Para la próxima clase

1. Investiga qué es **Bootstrap** y cómo puede mejorar el diseño frontend.
2. Revise la documentación sobre TypeScript:
 - <https://profile.es/blog/que-es-typescript-vs-javascript/>
 - <https://www.typescriptlang.org/docs/>
 - https://www.w3schools.com/typescript/typescript_intro.php
 - <https://medium.com/orbit-software/conceptos-basicos-de-typescript-6fd983610a38>
3. Explora la **carga diferida de módulos y componentes** en Angular para optimizar el rendimiento.

Referencias

- [Angular Oficial](#)

- [Conceptos básicos de TypeScript](#)
- [W3Schools: Introducción a TypeScript](#)